

Sentinel-AP — Autonomous Transaction Reliability Layer

Smart security guardrail between AI buyer agents and Razorpay

RAZORPAY BUILDATHON

Deterministic dual-gate middleware so agents can spend — safely, observably, and without stuck payouts.

PROBLEM

- ▶ **Hallucinated SKUs & over-budget spends** — LLM prompts are not a payment authorization layer.
- ▶ **Bank-rail outages** fail closed forever or retry blindly, creating stuck / duplicate money movement.
- ▶ **No audit trail** between agent intent and Razorpay Order — ops & compliance go dark.

SOLUTION — DUAL GATE

GATE 1 · HARD BLOCK

Deterministic policy: amount/daily caps, SKU lists, currency allow-list. Codes like
SKU_BLACKLISTED ,
AMOUNT_CAP_EXCEEDED .

GATE 2 · SOFT-FAIL

Pre-flight bank health (>95%). Degraded → QUEUED ; recover → Order → CLEARED . Never pay on a sick rail.

ARCHITECTURE

Agents never talk to Razorpay directly. Every spend intent hits Sentinel-AP first — auth, idempotency, dual gates, then Orders + Checkout + verify/webhooks.



TECH STACK

- FastAPI 1.2
- PostgreSQL
- Redis + ARQ
- Razorpay Orders
- Checkout.js
- Next.js 14
- Docker / Render / Vercel

LIVE DEMO

- WEB** <https://sentinel-ap.vercel.app>
- API** <https://sentinel-api-ecw9.onrender.com>
- DOCS** [../docs](#) · OpenAPI 1.2.0

ASK / IMPACT

Make **agentic commerce** production-safe on Razorpay: fewer unauthorized charges, zero stuck payouts on degraded rails, full decision audit for every intent.

WHY INTERNSHIP-READY

- ✓ **Idempotency-Key** — agent retries never duplicate Orders
- ✓ **Webhooks** — `payment.captured` receiver
- ✓ **OpenAPI 1.2** — typed agent/admin/payments surface
- ✓ **Audit + decision trail** — every gate outcome persisted
- ✓ **Checkout test mode** — live `order_...` + card 4111...
- ✓ **X-Request-Id + metrics** — observability built in